

Report – Boggle Solver

Overview

This assignment involved building a Boggle solver that finds valid words from a given grid and dictionary. The program uses a depth-first search (DFS) approach to check if words can be formed by connecting adjacent letters without reusing cells.

My approach was to first validate inputs, then implement DFS to explore all possible paths for each word. I also created and ran unit tests (using `unittest`) to check different scenarios, including normal cases, edge cases, and special tiles like "Qu" and "St". After confirming correctness, I used `flake8` and `black` to fix formatting issues while keeping the output unchanged.

Feedback Received

I had already properly implemented , tested and formatted the code before review, so i got a all clear to submit review from my reviewer.

Feedback Given

My reviewee had properly implemented and tested her code so i approved it after running and verifying it. I added comments of good and efficient practices in the code itself.

Improvements You Implemented

Since there were no improvements suggestions there were minimal improvements

1. Static Analysis Results

I ran `flake8` on my code and found several formatting issues. Most of them were related to indentation, spacing around operators, and extra or missing blank lines. I fixed all indentations to use 4 spaces, added proper spacing (like `a = b` instead of `a=b`), and removed trailing whitespace. I also shortened long lines and made sure the file ends with a newline.

After these fixes, the code passed `flake8` with no errors.

2. Regression Testing

For regression techniques, i did a end to end use of the whole program and reran all the tests after making any changes

3. Reflection

This task showed me that clean code is just as important as working code. Even if a program runs correctly, poor formatting makes it harder to read and maintain.

I learned how important peer feedback is to insure code compatibility with a larger system even when code is correct.

I also learned how useful tools like `flake8` and `black` are for catching and fixing issues automatically. In real-world projects, these tools help teams keep code consistent and professional.

Overall, I understand better now how code reviews and formatting standards play a role in professional software development.